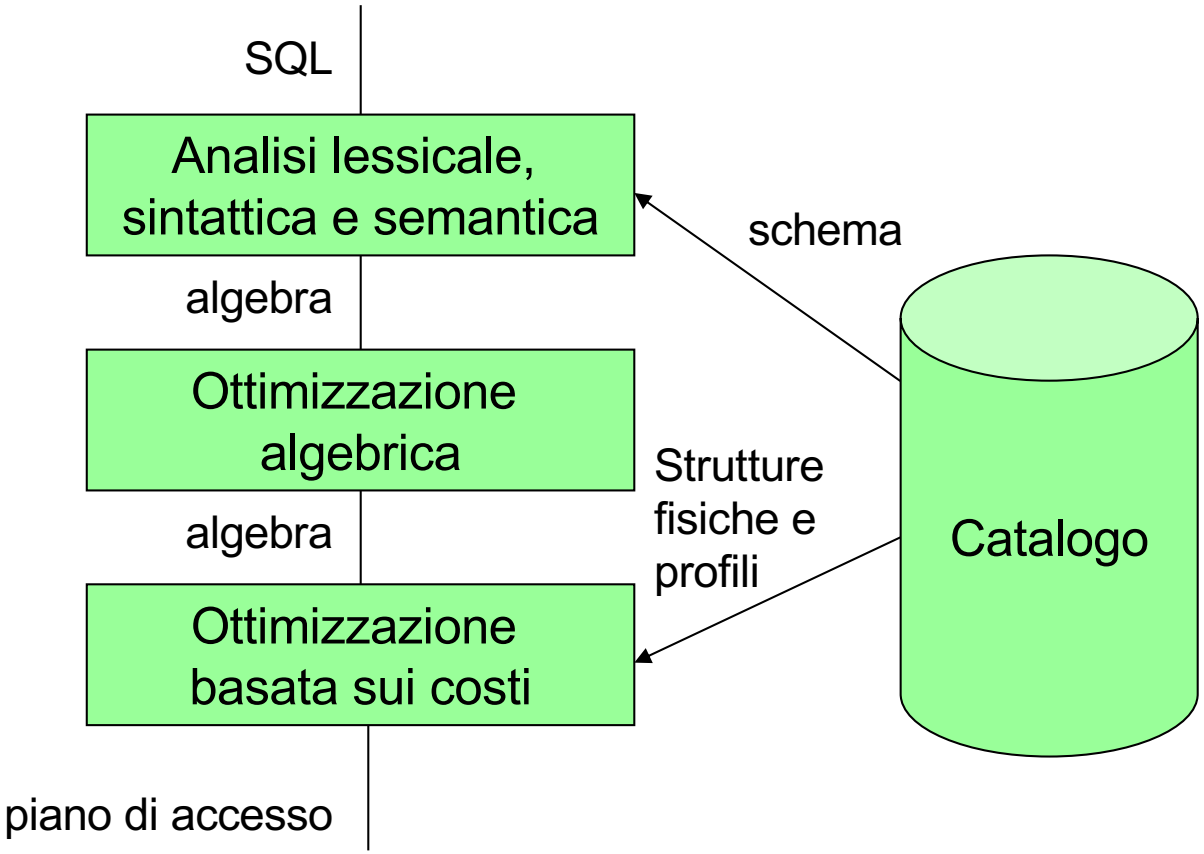
Il processo di esecuzione delle interrogazioni



Ottimizzazione basata sui costi

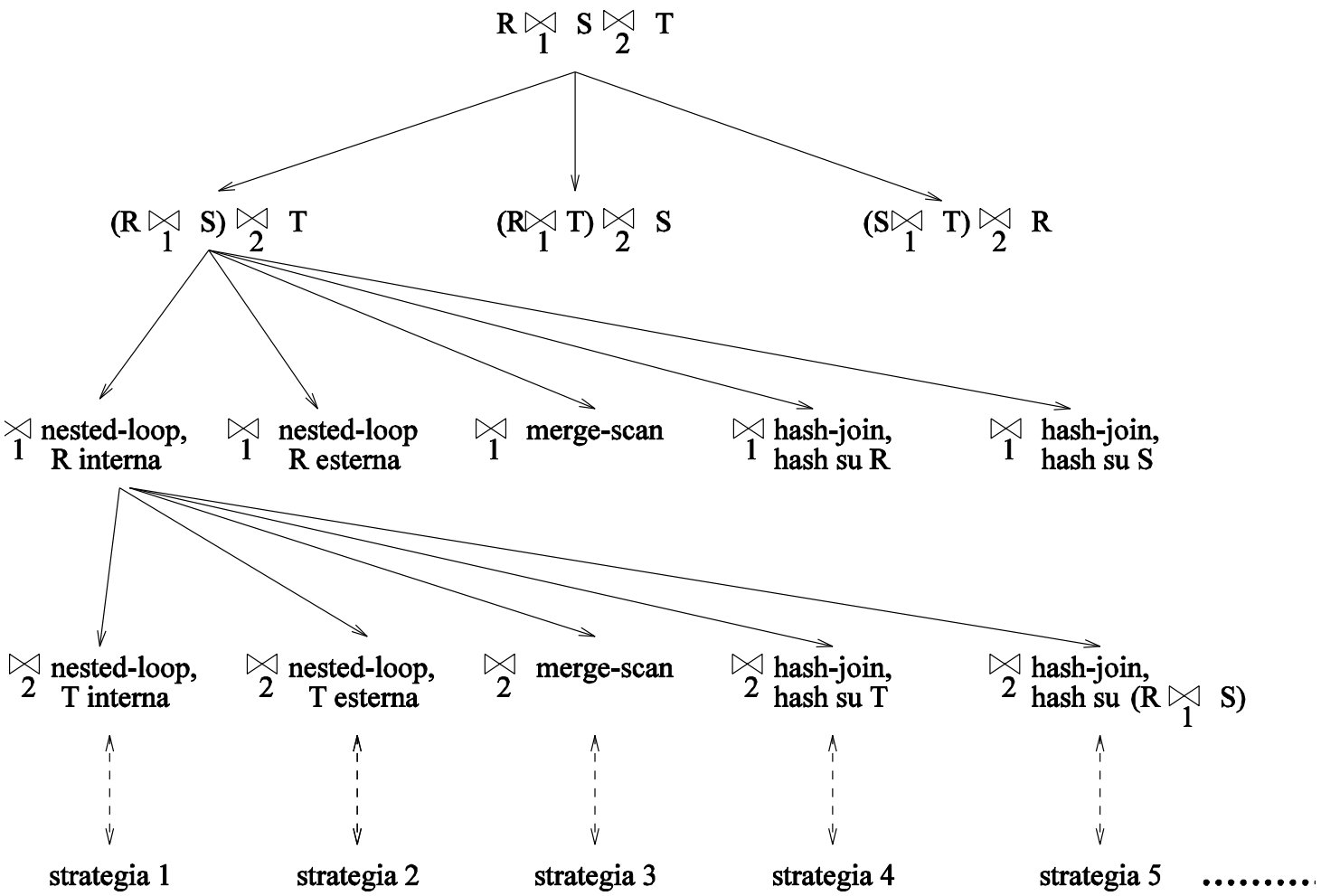
- Data un'espressione in algebra relazionale, come la si esegue?
- Un problema articolato, con scelte relative a:
  - operazioni da eseguire (es.: scansione o accesso diretto?)
  - ordine delle operazioni (es. join di tre relazioni; ordine?)
  - i dettagli del metodo (es.: quale metodo di join)
- Architetture parallele e distribuite aprono ulteriori gradi di libertà

Il processo di ottimizzazione

- Si costruisce un albero di decisione con le varie alternative ("**piani di esecuzione**")
- Si valuta il costo di ciascun piano (usando le statistiche o "profili" presenti nel catalogo)
- Si sceglie il piano di costo minore

- L'ottimizzatore trova di solito una "buona" soluzione, non necessariamente quella "ottima"

Un albero di decisione



Esempi

Impiegati(<u>Matricola</u>, Cognome, Nome, Ufficio)

100K impiegati, 10K uffici

Progetti(<u>Codice</u>, Titolo, Committente)

anche Titolo è chiave; 1000 progetti

Collaborazioni(<u>Impiegato</u>, <u>Progetto</u>, Ruolo)

500K collaborazioni

Due interrogazioni

- I dati dei progetti cui collaborano impiegati dell'ufficio 5103
- I dati degli impiegati che collaborano al progetto 'Marte'

Esempi

Impiegati(Matricola, Cognome, Nome, Ufficio); 100K impiegati, 10K uffici
Progetti(Codice, Titolo, Committente) anche Titolo è chiave; 1000 progetti
Collaborazioni(Impiegato, Progetto, Ruolo); 500K collaborazioni

- I dati dei progetti cui collaborano impiegati dell'Ufficio n. 5103

```
SELECT Progetti.*
FROM Impiegati JOIN Collaborazioni on Matricola=Impiegato
               JOIN Progetti on Progetto=Codice
WHERE Ufficio = 5103
```

- I dati degli impiegati che collaborano al progetto 'Marte'

```
SELECT Impiegati.*
FROM Impiegati JOIN Collaborazioni on Matricola=Impiegato
               JOIN Progetti on Progetto=Codice
WHERE Titolo = 'Marte'
```

Alla lavagna

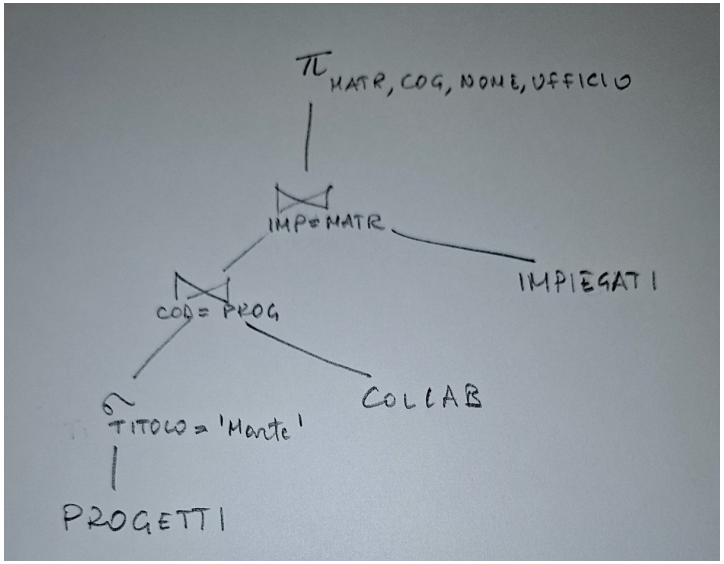
```
SELECT Progetti.*
FROM Impiegati JOIN Collaborazioni on Matricola=Impiegato
                JOIN Progetti on Progetto=Codice
WHERE Ufficio = 5103
```

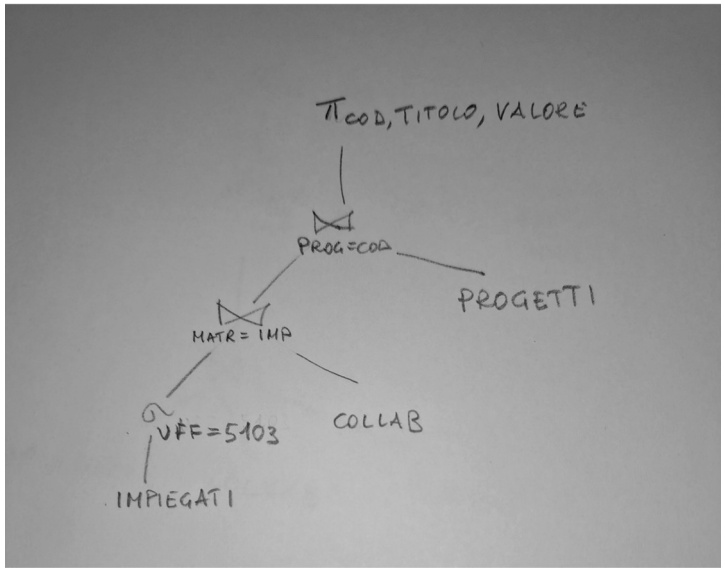
Dettagli (alla lavagna)

- Riscriviamo, anticipando selezioni (e anche proiezioni, ma tralasciamo)
- Scegliamo se fare scansioni sequenziali o accessi diretti
- Decidiamo quale join eseguire prima (quello più selettivo)
- Scegliamo il metodo per eseguire il join

Alla lavagna

```
SELECT Impiegati.*
FROM Impiegati JOIN Collaborazioni on Matricola=Impiegato
                JOIN Progetti on Progetto=Codice
WHERE Titolo = 'Marte'
```


π MATR, COG, NOME, UFFICIO
IMP=MATR
IMPIEGATI
COD=PROG
COLLAB
σ TITOLO='Marte'
PROGETTI


π COD, TITOLO, VALORE
PROG=COD
PROGETTI
MATR=IMP
σ UFF=5103
COLLAB
IMPIEGATI

Esercizio 3 del 9 aprile 2018

https://tagd.inf.uniroma3.it/compitiPDF/20180409bdIItestoEsoluz.pdf

Si consideri una base di dati con le relazioni (entrambe con indice sulla chiave primaria)

R1(A,B,C), R2(D,E,F)

Eseguendo le interrogazioni seguenti, su Postgres, si rilevano le seguenti scelte per l'operatore di join:

1.	`select *` `from R1 join R2 on C=D`	Hash join

2.	`select *`
	`from R1 join R2 on C=D`
	`where B>=41 AND B<=45`
		Nested loop join
		con accesso diretto alla relazione interna

Motivare ciò, valutando, per ciascuna delle due interrogazioni, il costo di un piano di esecuzione con hash join e uno con nested loop join (e che, per l'operazione 2, includa la selezione), supponendo che

- le relazioni abbiano N_1=2.000.000 ed N_2=4.000.000 ennuple, (con fattore di blocco f_1=20 e f_2=40)
- l'attributo B in R1 abbia circa 200.000 valori diversi (compresi fra 1 e 200.000 e distribuiti uniformemente)
- entrambi gli indici abbiano p=4 livelli (radice e foglie incluse) e fattore di blocco massimo f_i=50
- l'operazione possa contare su un numero di pagine di buffer pari a circa q=500.

Soluzione

- Alla lavagna
- E poi vediamo
  https://tagd.inf.uniroma3.it/compitiPDF/20180409bdIItestoEsoluz.pdf

Pipelining vs materializzazione

- Due alternative per le interrogazioni nei sottoalberi:
  - **pipelining**: le ennuple sono "utilizzate" dal nodo superiore a mano a mano che vengono prodotte (anzi, vengono prodotte a richiesta):
    - vantaggio: non dovendo salvare i risultati intermedi, riduce i costi di I/O
    - svantaggio: se i risultati intermedi vengono riutilizzati più volte, è necessario ricalcolarli (ad esempio, il ciclo interno di un nested loop)
  - **materializzazione**: l'intero risultato intermedio viene prodotto e memorizzato, prima di essere utilizzato

Progettazione fisica

- La fase finale del processo di progettazione di basi di dati
- input
  - lo schema logico e informazioni sul carico applicativo
- output
  - schema fisico, costituito dalle definizione delle relazioni con le relative strutture fisiche (e molti parametri, spesso legati allo specifico DBMS)

Strutture fisiche nei DBMS relazionali

- Struttura primaria:
  - disordinata (heap, "unclustered")
  - ordinata ("clustered"), anche su una pseudochiave
  - hash ("clustered"), anche su una pseudochiave, senza ordinamento
  - clustering di più relazioni (vedremo in un esercizio)
- Indici (densi/sparsi, semplici/composti):
  - ISAM (statico), di solito su struttura ordinata
  - B-tree (dinamico)
  - Indici hash (secondario, poco dinamico)

Strutture fisiche in alcuni DBMS

- Oracle:
  - struttura primaria
    - file heap
    - "hash cluster" (cioè struttura hash)
    - cluster (anche plurirelazionali) anche ordinati (con B-tree denso)
  - indici secondari di vario tipo (B-tree, bit-map, funzioni)
- DB2:
  - primaria: heap o ordinata con B-tree denso
  - indice sulla chiave primaria (automaticamente)
  - indici secondari B-tree densi
- SQL Server:
  - primaria: heap o ordinata con indice B-tree sparso
  - indici secondari B-tree densi

Strutture fisiche in alcuni DBMS, 2

- Ingres (molti anni fa):
  - file heap, hash, ISAM (ciascuno anche compresso)
  - indici secondari
- Informix (molti anni fa):
  - file heap
  - indici secondari (e primari [cluster] ma non mantenuti)

Definizione degli indici SQL

- Non è standard, ma presente in forma simile nei vari DBMS
  - `create` [`unique`] `index` *IndexName* `on` *TableName*(*AttributeList*)
  - `drop index` *IndexName*

- `unique` è un po' contraddittorio (e metodologicamente sbagliato)

Postgres, comandi su strutture fisiche

CREATE [UNIQUE] INDEX [name] ON table [USING method]
({ column | (expression) } [COLLATE collation] [opclass] [ASC | DESC] [NULLS { FIRST | LAST }] [, ...]) [WITH (storage_parameter = value [, ...])] [TABLESPACE tablespace] [WHERE predicate]

CLUSTER [VERBOSE] table_name [USING index_name]

VACUUM [({ FULL | FREEZE | VERBOSE | ANALYZE } [, ...])] [table [(column [, ...])]]

- Guardare la documentazione

Progettazione fisica nel modello relazionale

- La caratteristica comune dei DBMS relazionali è la disponibilità degli indici:
  - la progettazione fisica spesso coincide con la scelta degli indici (oltre ai parametri strettamente dipendenti dal DBMS)
- Le chiavi (primarie) delle relazioni sono di solito coinvolte in selezioni e join: molti sistemi prevedono (oppure suggeriscono) di definire indici sulle chiavi primarie
- Altri indici vengono definiti con riferimento ad altre selezioni o join "importanti"
- Se le prestazioni sono insoddisfacenti, si "tara" il sistema aggiungendo o eliminando indici o modificando le strutture primarie (attività di "tuning")
- È utile verificare se e come gli indici sono utilizzati con il comando SQL `show plan` oppure `explain`

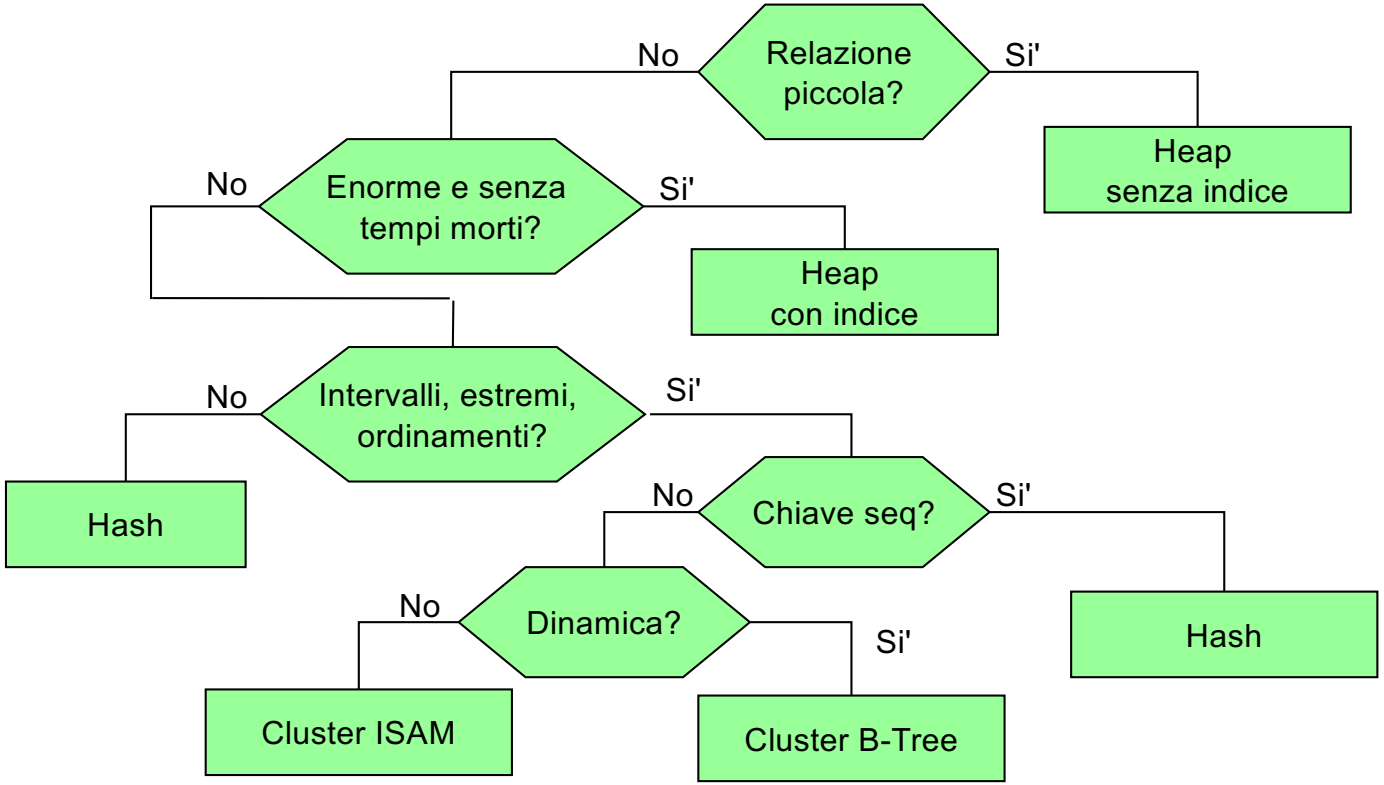
Progettazione fisica: euristiche suggerite da Informix

- Non creare indici su relazioni piccole (<200 ennuple)
- non creare indici su campi con pochi valori (se proprio servono, che siano primari)
- creare indici su campi con selezioni
- per i join: creare indici sulla relazione più grande

Scelta della struttura secondo Shasha

D. Shasha. Database Tuning: a principled approach. Prentice-Hall, 1992

D. Shasha, P. Bonnet Database Tuning: Principles, Experiments, and Troubleshooting Techniques Morgan Kaufmann 2002



Esercizio 3 del 9 aprile 2018, ancora

https://tagd.inf.uniroma3.it/compitiPDF/20180409bdIItestoEsoluz.pdf

Si consideri una base di dati con le relazioni (entrambe con indice sulla chiave primaria)

R1(A,B,C), R2(D,E,F)

Eseguendo le interrogazioni seguenti, su Postgres, si rilevano le seguenti scelte per l'operatore di join:

1.	`select *` `from R1 join R2 on C=D`	Hash join

2.	`select *`
	`from R1 join R2 on C=D`
	`where B>=41 AND B<=45`
		Nested loop join
		con accesso diretto alla relazione interna

Motivare ciò, valutando, per ciascuna delle due interrogazioni, il costo di un piano di esecuzione con hash join e uno con nested loop join (e che, per l'operazione 2, includa la selezione), supponendo che

- le relazioni abbiano N_1=2.000.000 ed N_2=4.000.000 ennuple, (con fattore di blocco f_1=20 e f_2=40)
- l'attributo B in R1 abbia circa 200.000 valori diversi (compresi fra 1 e 200.000 e distribuiti uniformemente)
- entrambi gli indici abbiano p=4 livelli (radice e foglie incluse) e fattore di blocco massimo f_i=50
- l'operazione possa contare su un numero di pagine di buffer pari a circa q=500.

- Abbiamo visto sulla carta, ma possiamo toccare con mano, su Postgres (o altro sistema)

Esempio reale con Postgres

```
set search_path to "20260327proveOttimizzazione";

drop table if exists r1;
create table r1 (     a numeric not null primary key,
                      b numeric,
                      c numeric);
drop table if exists r2;
create table r2 (     d numeric not null primary key,
                      e numeric,
                      f numeric);
alter table r1 add constraint fk1 foreign key (c) references r2 (d);
```

Esempio reale con Postgres (2)

- popoliamo le strutture
  - con insert (un po' noioso e non scalabile)
  - con copy da file excel
  - con script,vediamo (guardare il manuale per capire il significato e per applicare varianti)

Esempio, continua

```
create or replace function loadr2() returns void as $$
begin
    for i in 1..4000000 loop insert into r2
    values(i,floor(random()*20+1),floor(random()*1000)+1); end loop;
end;
$$ language plpgsql;
select loadr2();
create or replace function loadr1() returns void as $$
begin
    for i in 1..2000000 loop insert into r1
    values(i,floor(random()*200000+1),floor(random()*4000000)+1); end loop;
end;
$$ language plpgsql;
select loadr1();
```

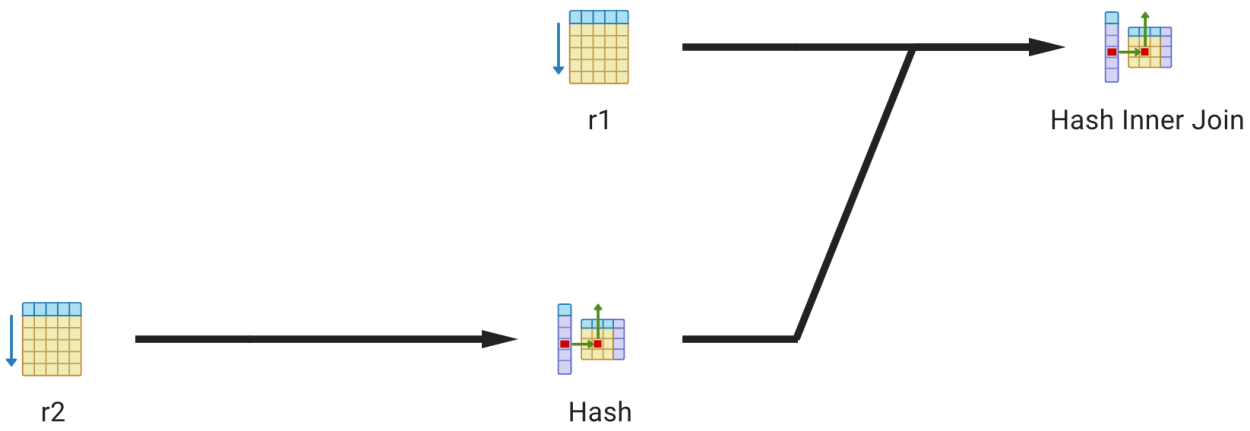
```
vacuum analyze;
```

Esempio con Postgres

```
1  select * from r1 join r2 on c=d
2
3
4
```

Data Output Messages Explain × Notifications

Graphical Analysis Statistics



Vediamo quale join viene scelto

```
select *
from r1 join r2 on C=D
```

Possiamo forzare il metodo di join (inibendo gli altri); eseguire varie prove a conferma di quanto scritto nel compito

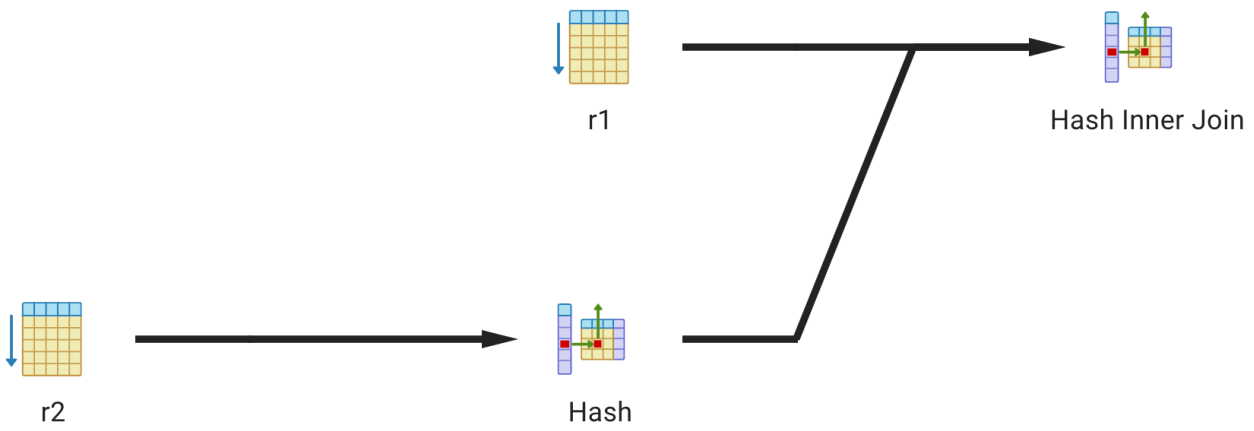
```
set enable_hashjoin=0; -- 0 disabilita 1 abilita
set enable_mergejoin=0;
set enable_nestloop=0
set max_parallel_workers = 0
       ---disabilita parallelismo (in un certo
senso)

explain analyze
select * from r1 join r2 on c=d;
```

Esempio con Postgres (2)

```
1  select * from r1 join r2 on c=d
2
3
4
```

Data Output Messages Explain × Notifications

Graphical Analysis Statistics



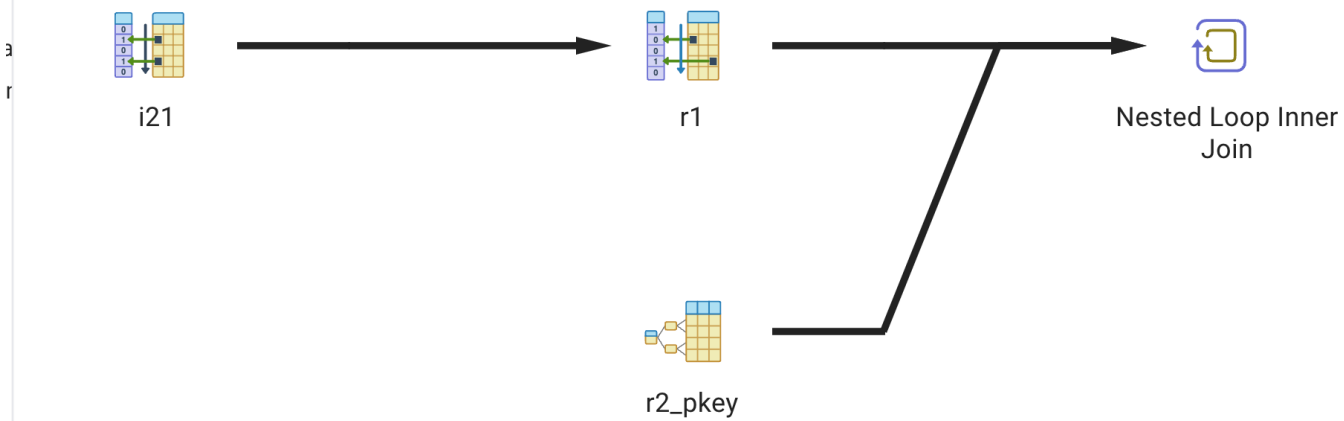
Esempio con Postgres (3)

Query Query History

```
1  create index i21 on R1(B);
2  select *
3  from r1 join r2 on c=d
4  where B>=40 AND B<=45
```

Data Output Messages Explain × Notifications

Graphical Analysis Statistics



Risultati

- Conferma di quanto visto sulla carta
  - interrogazione 1
    - conviene hash-join, poi merge-join, ultimo nested-loop
  - Interrogazione 2
    - conviene nested-loop, poi hash-join, ultimo merge-join
  - Interrogazione 2bis
    - conviene di gran lunga nested-loop, poi hash-join, ultimo merge-join

Volendo approfondire (ma non lo facciamo)

- Dimensione del blocco
  SHOW block_size;
  SELECT current_setting('block_size');
- Spazio occupato da un valore di un certo tipo
  SELECT pg_column_size(1::numeric);
- Spazio occupato da una tabella in MB
  SELECT pg_size_pretty (pg_table_size('r1'));
- Spazio occupato dagli indici di una tabella in MB
  SELECT pg_size_pretty (pg_indexes_size('r2'));
- Numero di ennuple e numero di blocchi di una tabella
  ANALYZE VERBOSE r1

https://www.postgresql.org/docs/11/functions-admin.html
https://www.postgresql.org/docs/11/runtime-config-preset.html
https://www.postgresql.org/docs/11/sql-show.html
(https://www.postgresqltutorial.com/postgresql-database-indexes-table-size/)
(https://www.postgresql.org/docs/11/sql-analyze.html)

Nell'esempio

- Dimensione del blocco 8 KB
- Dimensione di un attributo di tipo numerico 8 B
- Dimensione di una ennupla (tre attributi numerici) 24 B
- Fattore di blocco (calcolato da noi): 8192 / 24 = ca 333
- Con ANALYZE VERBOSE
  - Numero di ennuple 4.000.000
  - Numero di blocchi 25.469

– Il fattore di blocco è quindi molto più piccolo (157), perché un blocco contiene molte informazioni di servizio

(http://www.interdb.jp/pg/pgsql01.html)

Progetto

- Sperimentare le strutture fisiche di un DBMS; definire alcune relazioni (ad esempio tre) e alcune interrogazioni (due o tre) che prevedano selezioni, proiezioni e join.
- Utilizzare relazioni di dimensioni sufficientemente grandi da rendere conveniente l'uso degli indici (si suggerisce di generare, con opportuni progammi, dati sintetici casuali).
- Mostrare, con riferimento al DBMS scelto (DB2, PostgresSQL, Oracle, etc.) il comportamento del sistema (in termini di piano di esecuzione delle interrogazioni), in presenza e assenza di indici e prima e dopo l'aggiornamento delle statistiche.
- Sintetizzare il tutto in una relazione **(in txt o pdf, all'interno di un file zip, da consegnare prima della prova intermedia o dell'esame)** di alcune pagine (con allegati i test), che permetta di comprendere il lavoro svolto e i risultati ottenuti.
- **Il progetto non viene valutato ma deve essere consegnato.** Il lavoro può essere svolto in collaborazione, ma gli esperimenti debbono essere, almeno in parte, individuali e le consegne debbono essere individuali.

Altri esercizi

https://tagd.inf.uniroma3.it/20252026/progettihomework.html

- Esercizi proposti, nella lezione del 1/04 potremo discuterli

7. join con nested loop e buffer, **esercizio 1 del 9/04/2018**
8. join con indice B-tree e buffer, **esercizio 3 del 28/03/2014**
9. valutazione di alternative per la struttura fisica, **esercizio 2 del 9/04/2018**
10. cluster plurirelazionali, **esercizio 5 del 28/06/2016**
11. ordinamento non mantenuto, **esercizio 2 del 10/04/2017**
12. esercizio su mergesort con molti buffer, **esercizio 5 del 26/02/2013**
13. esercizio su interrogazione aggregativa con molti buffer, **esercizio 3 del 21/06/2018**